

PRINCIPIILE SOLID

Tehnici avansate de programare

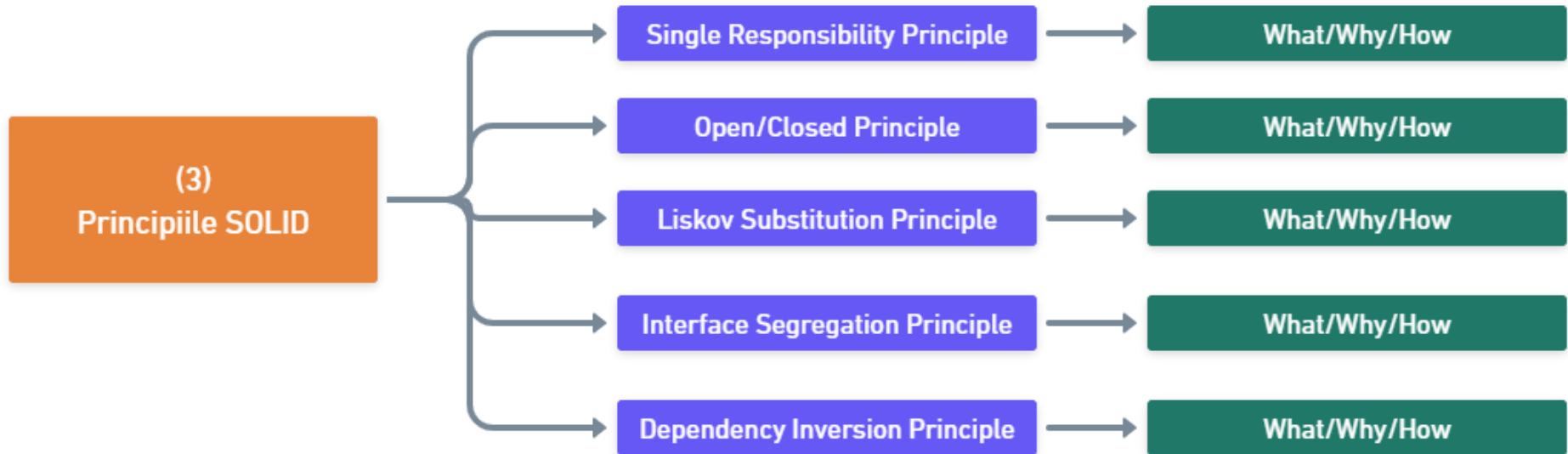
Nicoleta Scrimint



Facultatea de
MATEMATICĂ



Agenda



Made with Whimsical



SOLID

SRP

Single Responsibility Principle

OCP

Open/Closed Principle

LSP

Liskov Substitution Principle

ISP

Interface Segregation Principle

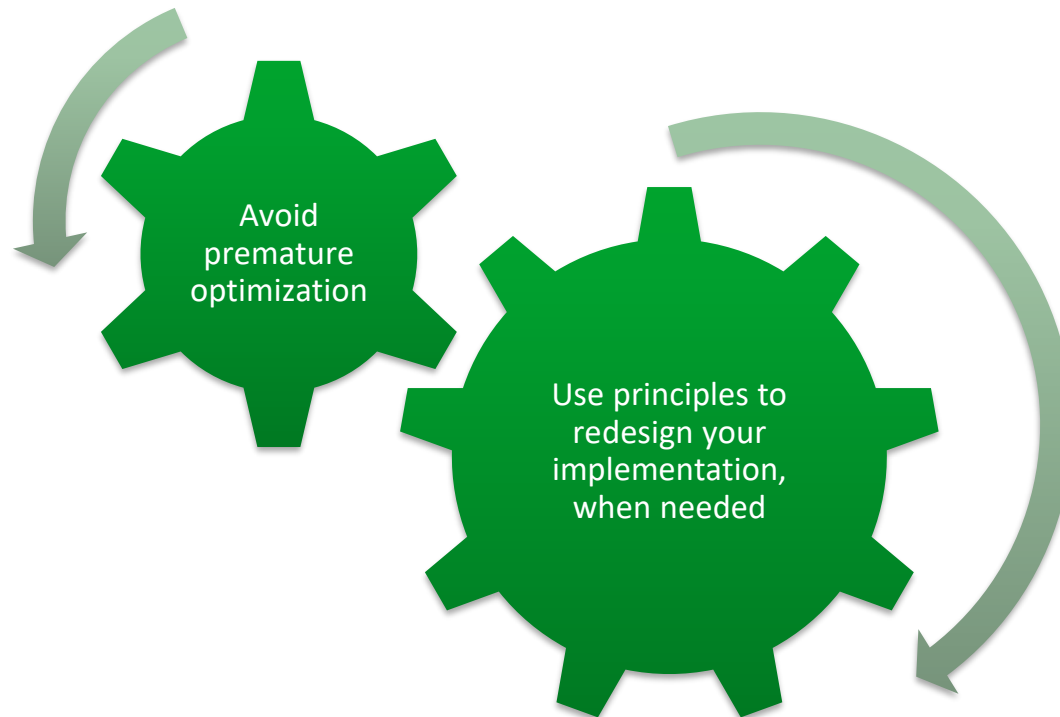
DIP

Dependency Inversion Principle



PDD

- Pain Driven Development 😞



Single Responsibility Principle

SRP

Single Responsibility Principle

Open/Closed Principle

Liskov Substitution Principle

Interface Segregation Principle

Dependency Inversion Principle



Single Responsibility Principle

- Each software module should have **one and only one** reason to change.
- This does not mean that a class or function can do only one thing, actually what this principle contemplates is that changes to an element of the code must be related to each other under the same concept, actor or context.



Single Responsibility Principle

- What is a **Responsibility**?

Examples:



Persistence



Logging

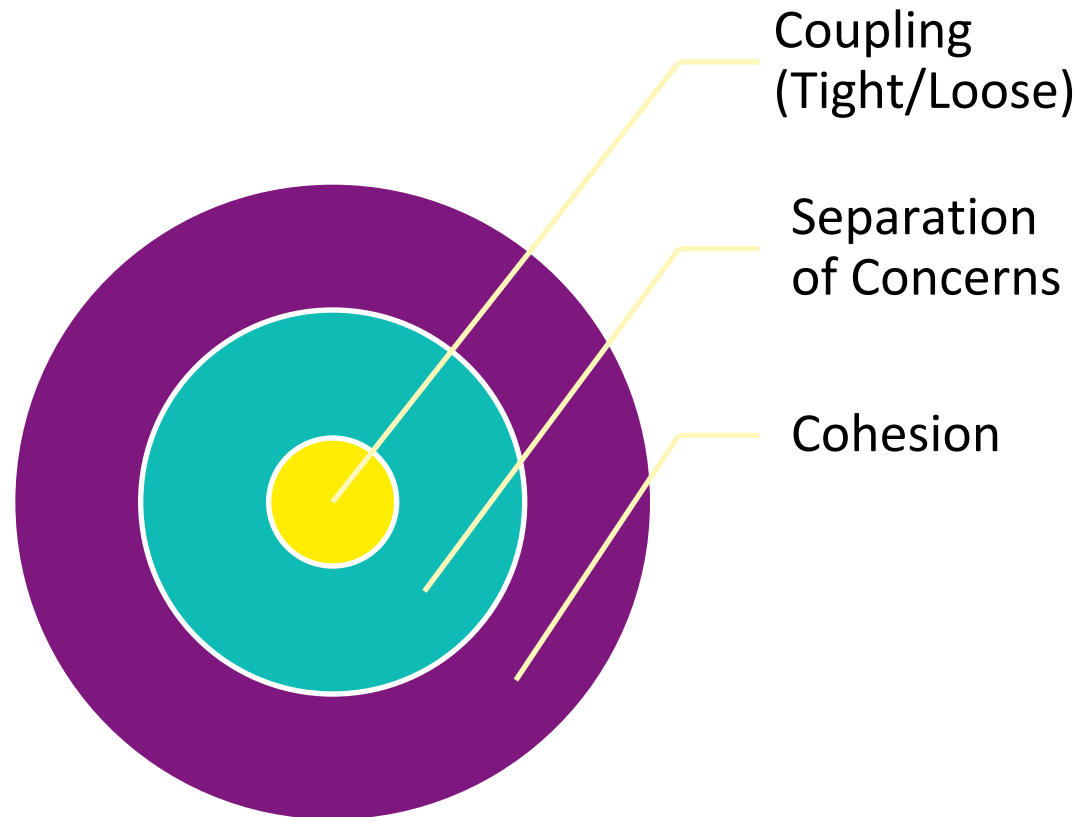


Validation



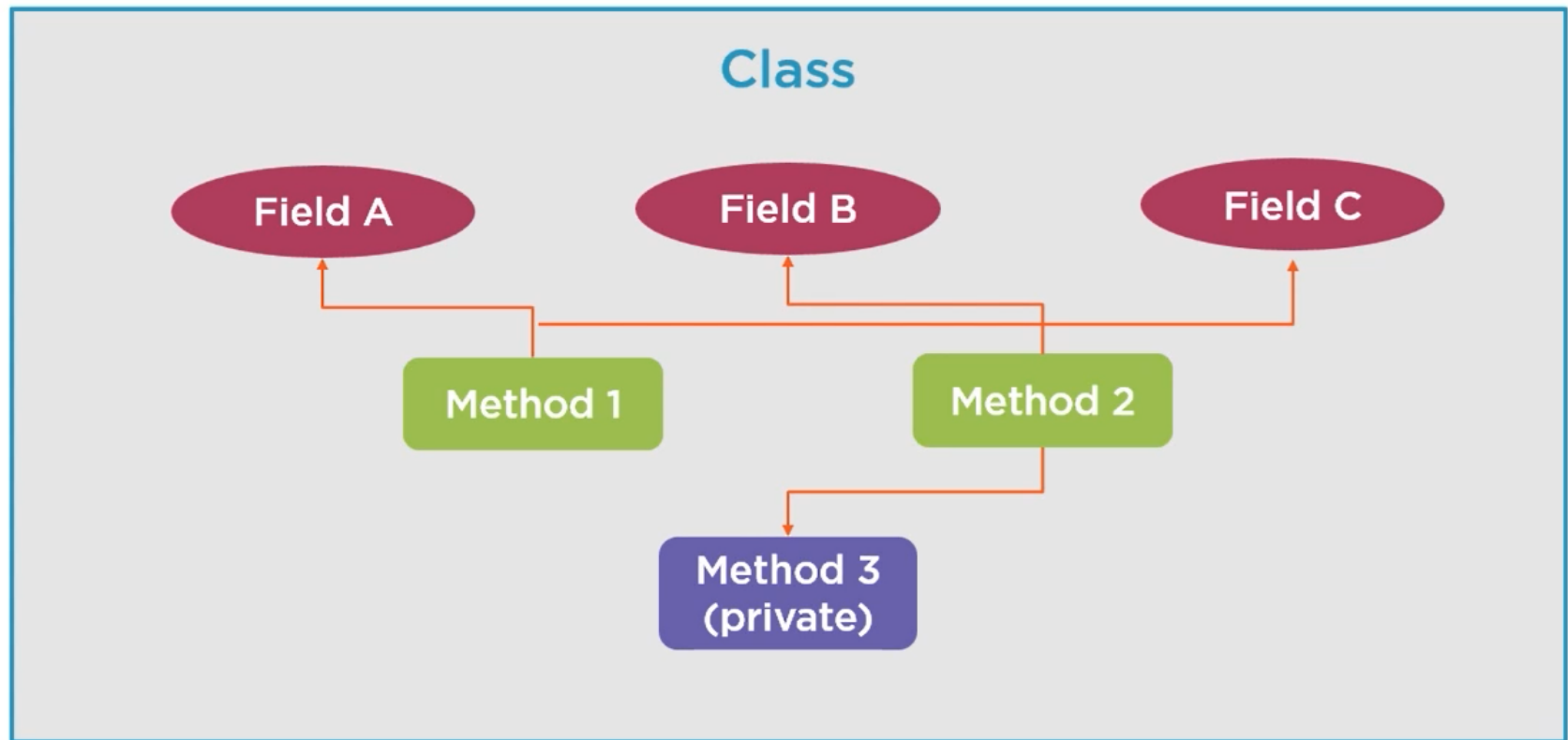
Business Logic

Single Responsibility Principle



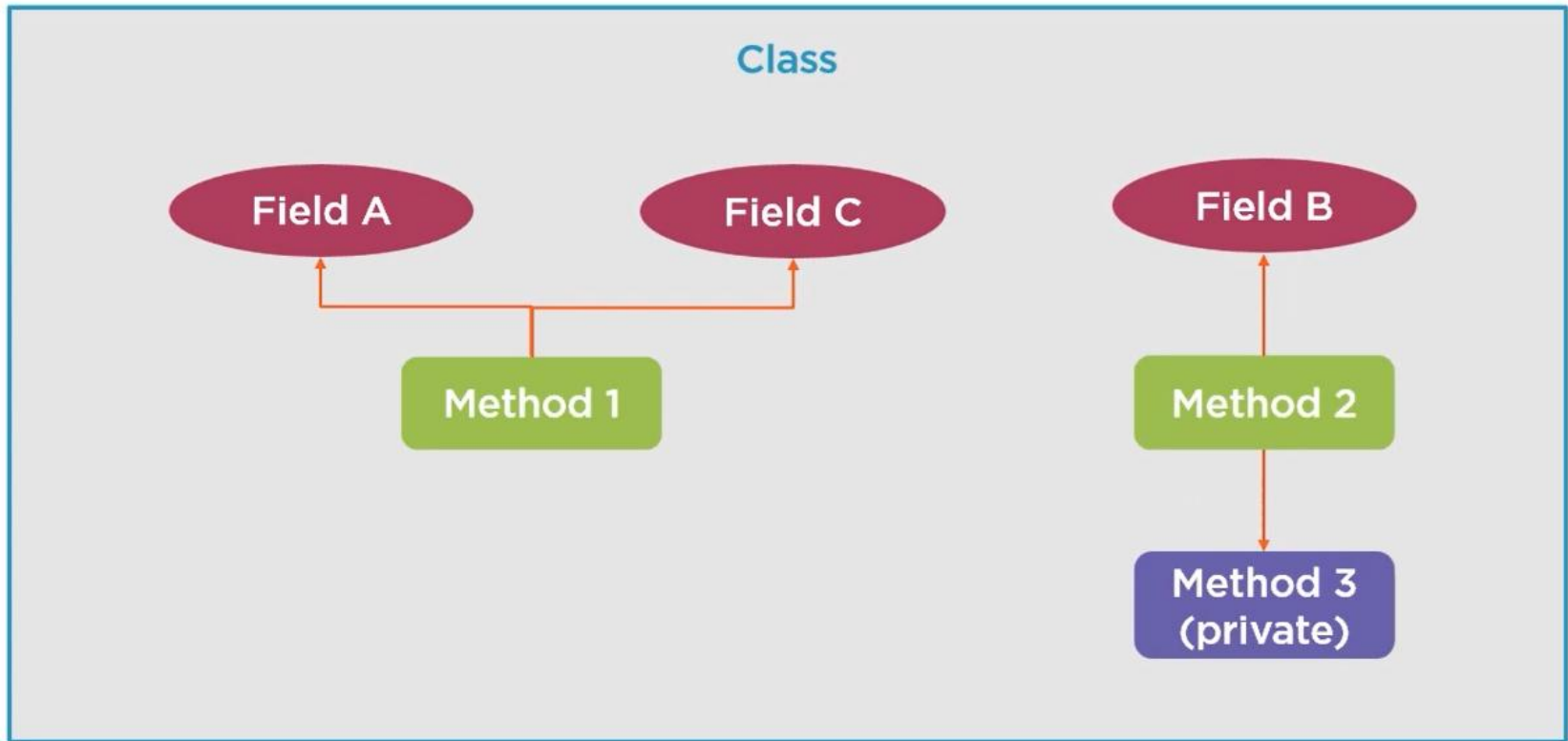
Single Responsibility Principle

Class Cohesion



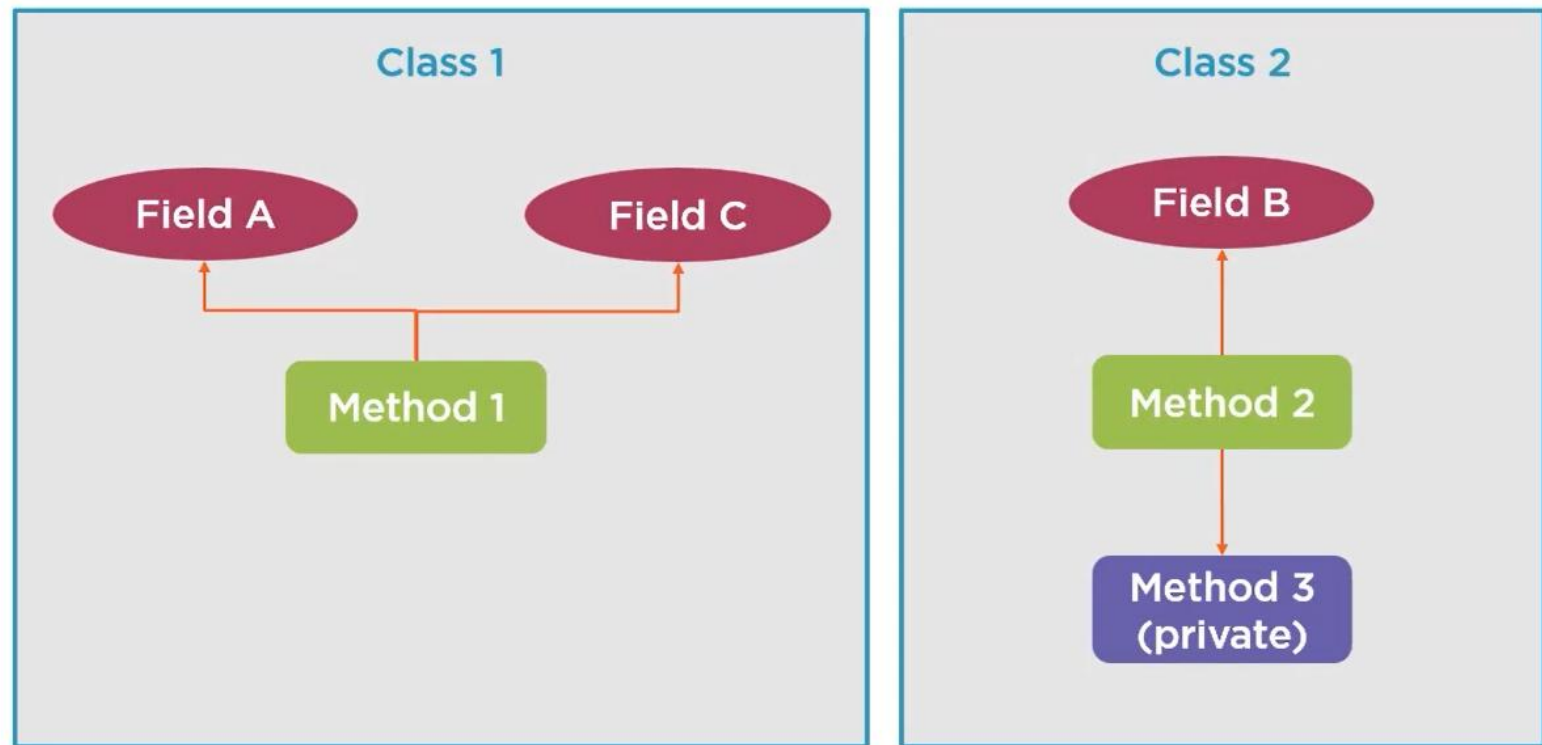
Single Responsibility Principle

Class Cohesion (Low)



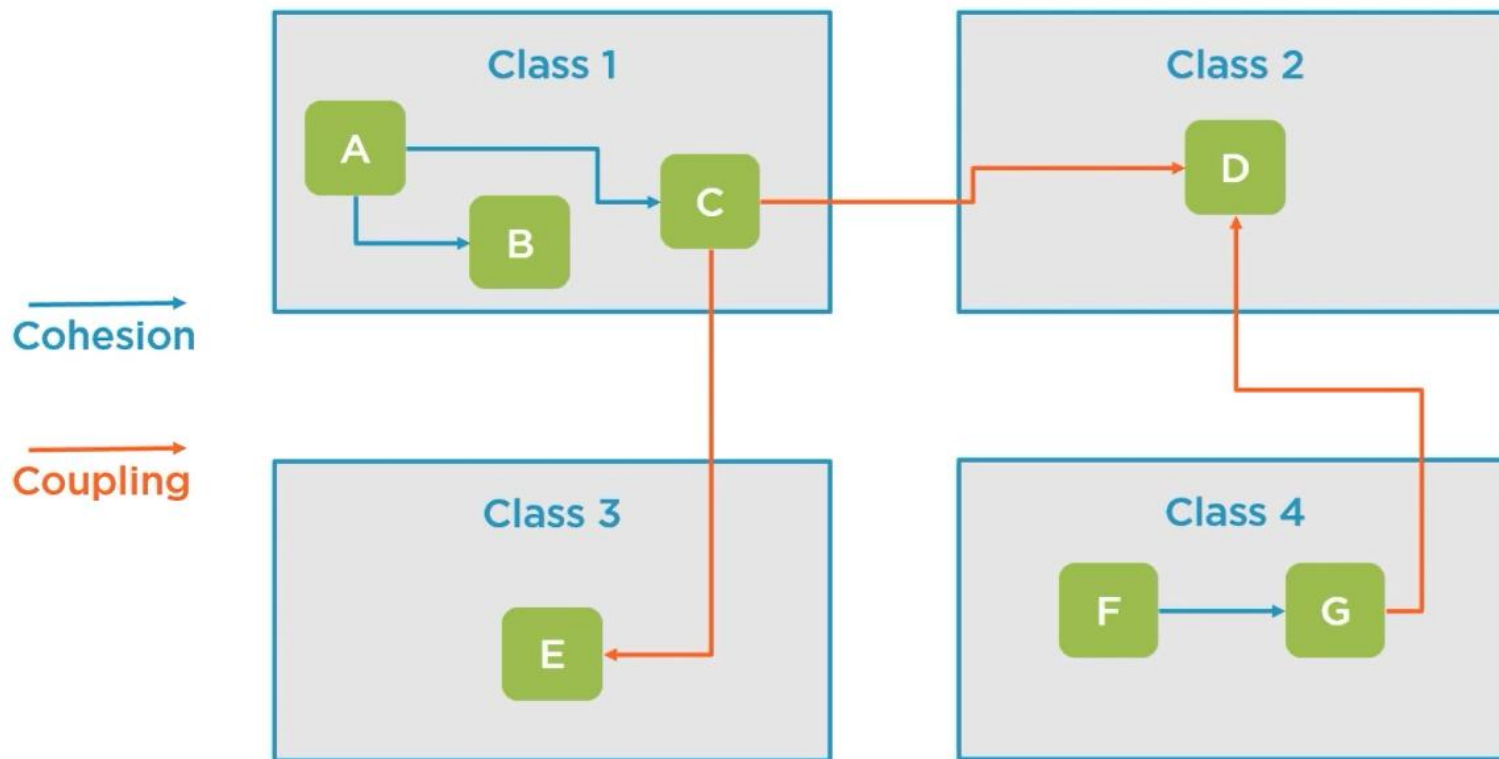
Single Responsibility Principle

Class Cohesion (High)



Single Responsibility Principle

Class Coupling and Cohesion

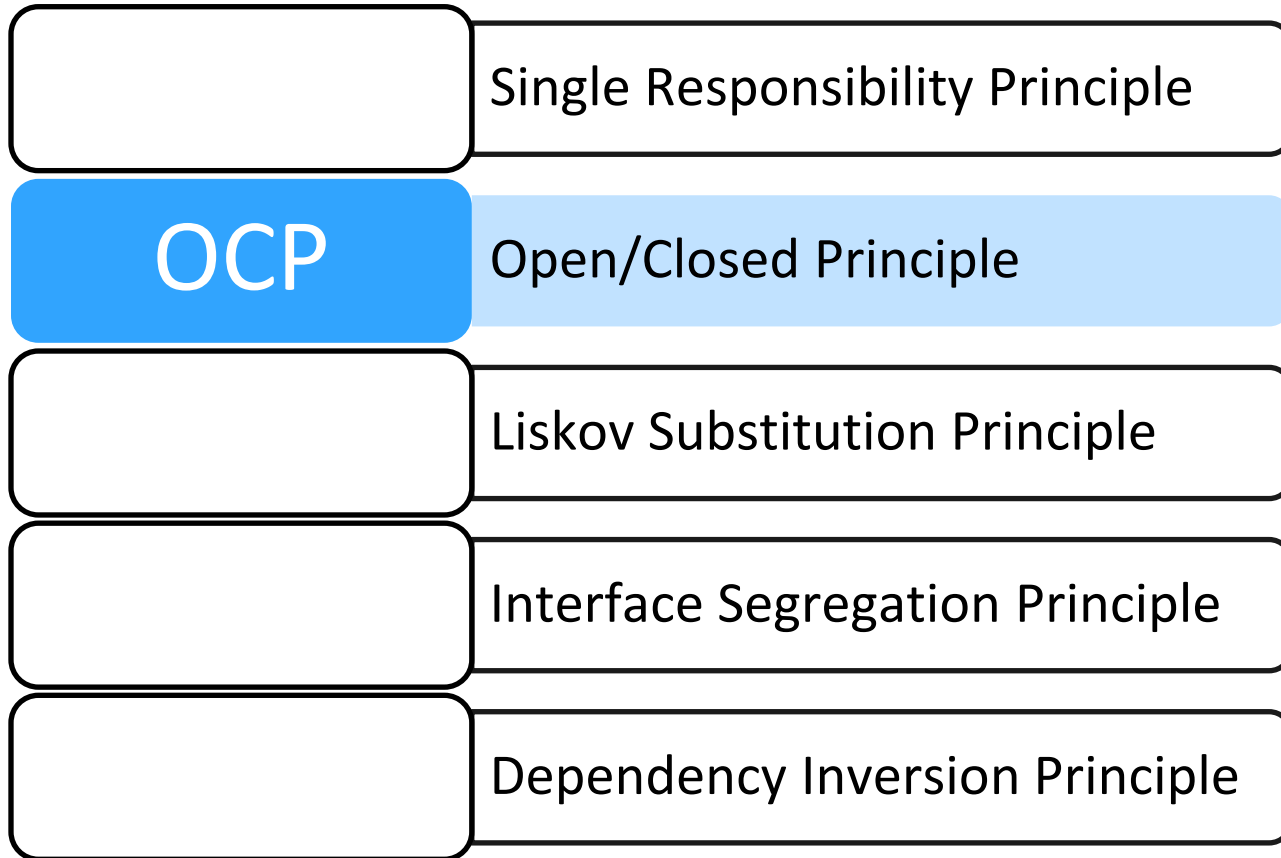


Single Responsibility Principle

- Some examples of responsibilities to consider that may need to be separated include:
 - Persistence
 - Validation
 - Notification
 - Error Handling
 - Logging
 - Formatting
 - Parsing
 - Mapping



Open/Closed Principle



Open/Closed Principle

- Software entities (classes, modules, functions, etc.) should be **open for extension**, but **closed for modifications**.



Open/Closed Principle

Open / Closed

Open to extension

New behavior can be added
in the future

Code that is closed to extension has
fixed behavior

Closed to modification

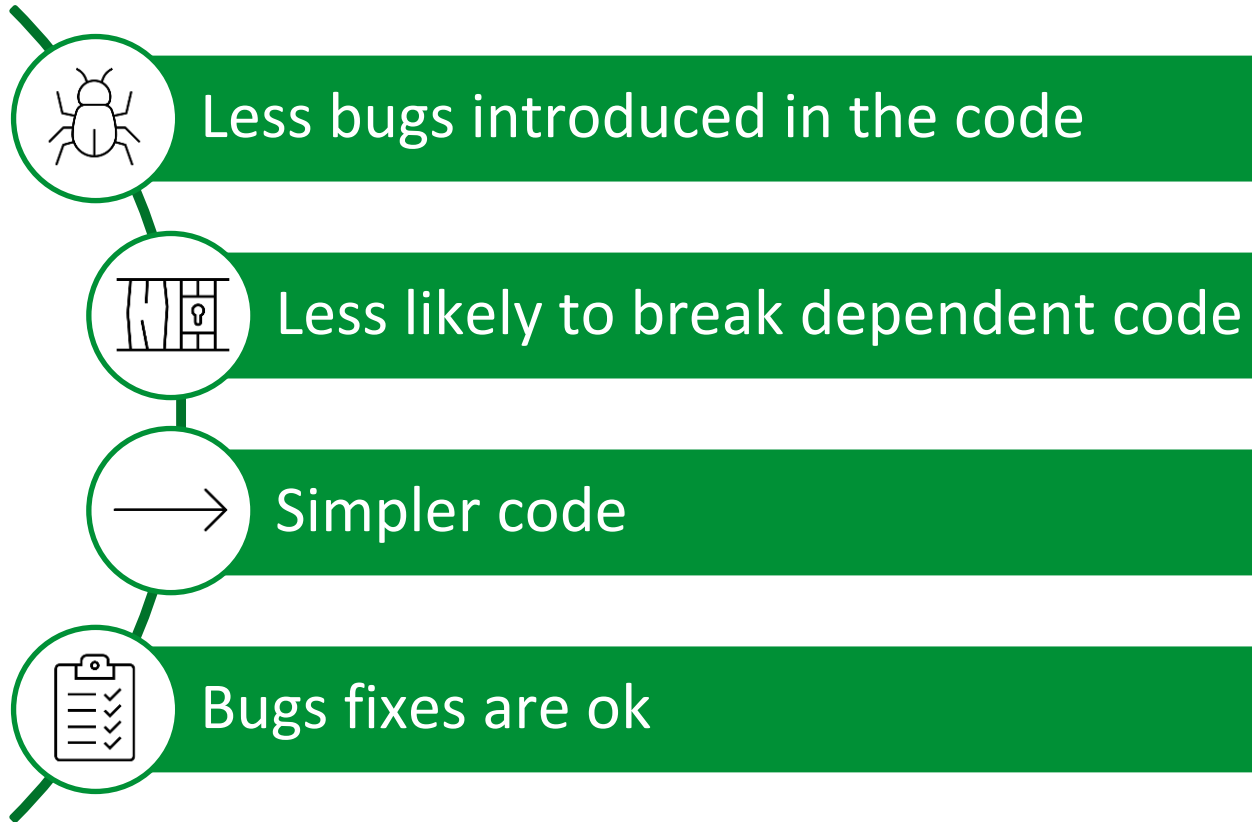
Changes to source or binary code are
not required

The only way to change the behavior of
code that is closed to extension is to
change the code itself

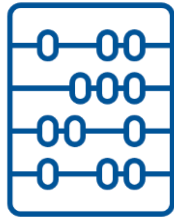


Open/Closed Principle

Why Should Code Be Closed to Modification?

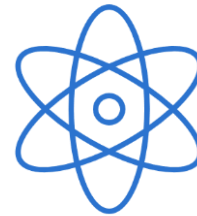


Open/Closed Principle



Balance abstraction
and concreteness

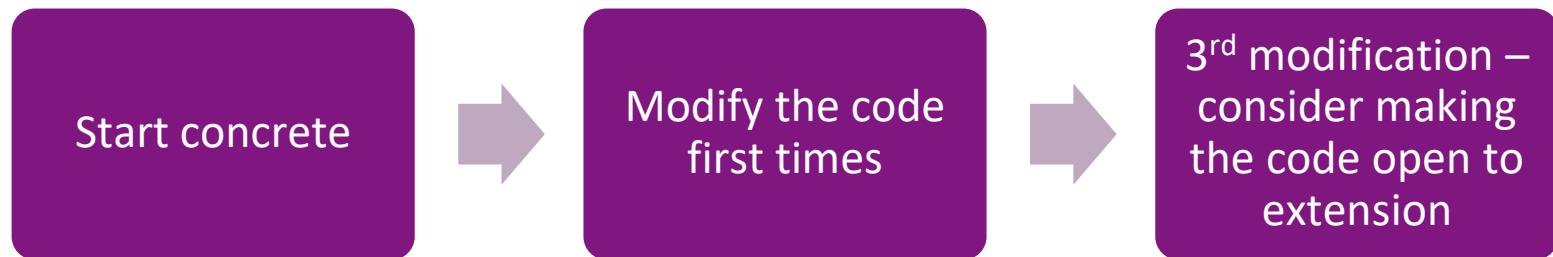
Abstractions adds
complexity



Predict where
variation is needed and
apply abstraction as
needed

Open/Closed Principle

How Can You Predict Future Changes?

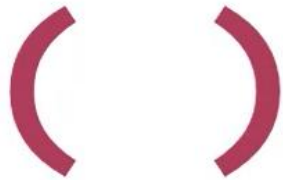


Open/Closed Principle

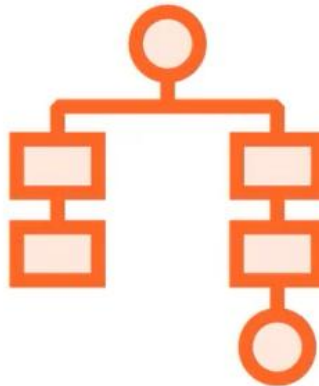
- To be applied for applications which are consumed by external parties and thus, are hard to change:
 - User interfaces
 - API services
 - Console applications as tooling
 - Reusable libraries
- YAGNI principle overrules OCP when developers have full control over how the code is being used
 - Prevents possible unnecessary code maintenance
 - Code can evolve with code refactoring when there is a real need

Open/Closed Principle

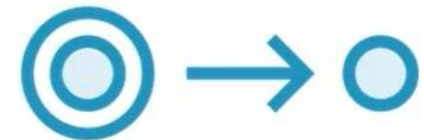
Typical Approaches to OCP



Parameters



Inheritance



Composition /
Injection

Open/Closed Principle

```
public class DoOneThing
{
    public void Execute()
    {
        Console.WriteLine("Hello world.");
    }
}
```

```
public class DoOneThing
{
    public void Execute(string message)
    {
        Console.WriteLine(message);
    }
}
```

```
public class DoOneThing
{
    public virtual void Execute()
    {
        Console.WriteLine("Hello world.");
    }
}
public class DoAnotherThing: DoOneThing
{
    public override void Execute()
    {
        Console.WriteLine("Goodbye world!");
    }
}
```

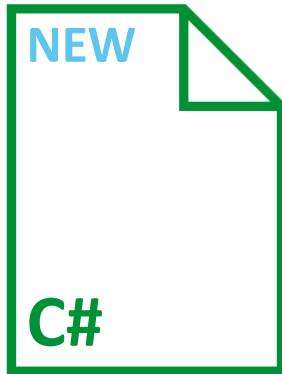
```
public class DoOneThing
{
    private readonly MessageService _messageService;
    public DoOneThing(MessageService messageService)
        => _messageService = messageService;

    public void Execute()
    {
        Console.WriteLine(_messageService.GetMessage());
    }
}
```



Open/Closed Principle

Why Use a New Class?



Design class to suit problem at hand

Nothing in current system depends on it

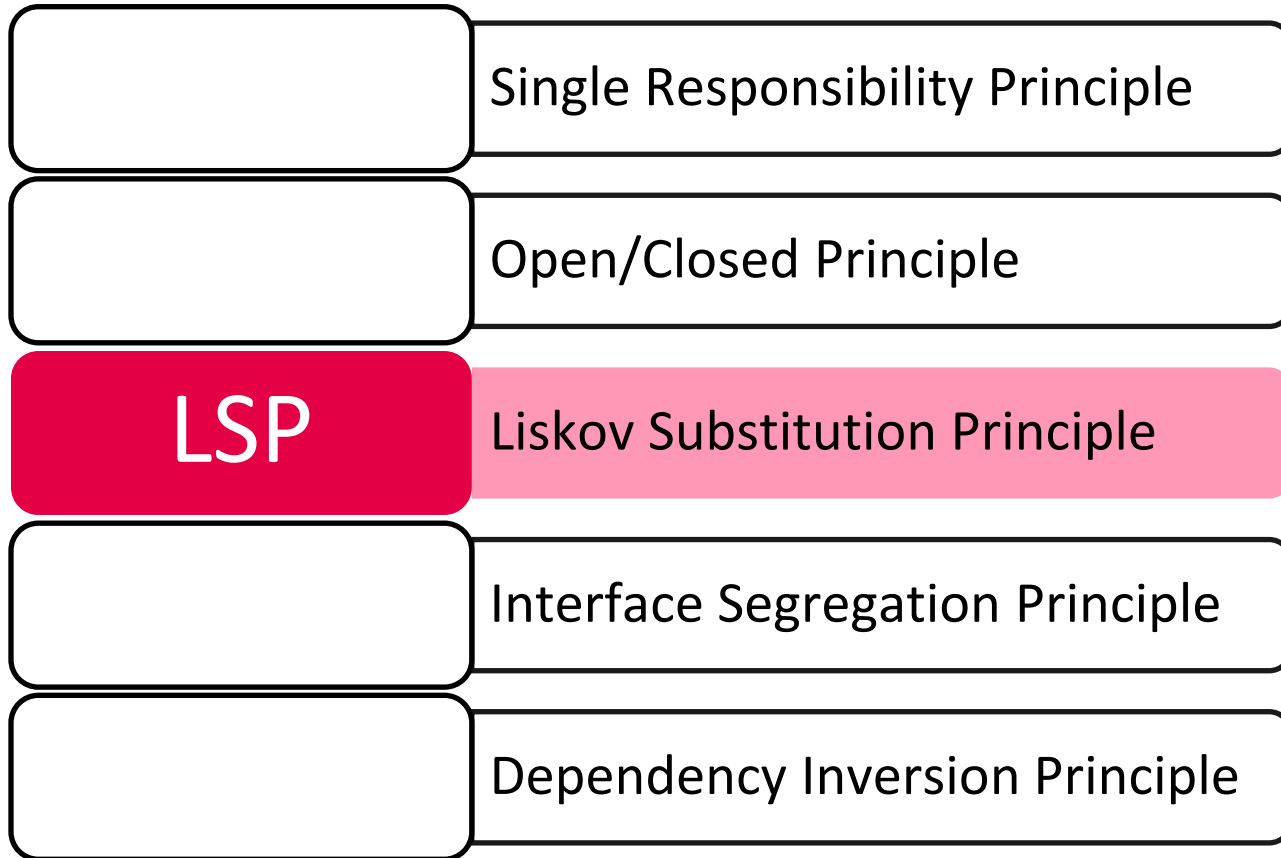
Can add behavior without touching existing code

Can follow Single Responsibility Principle

Can be unit-tested

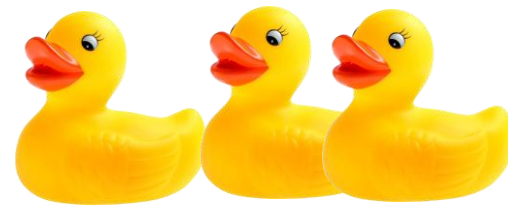
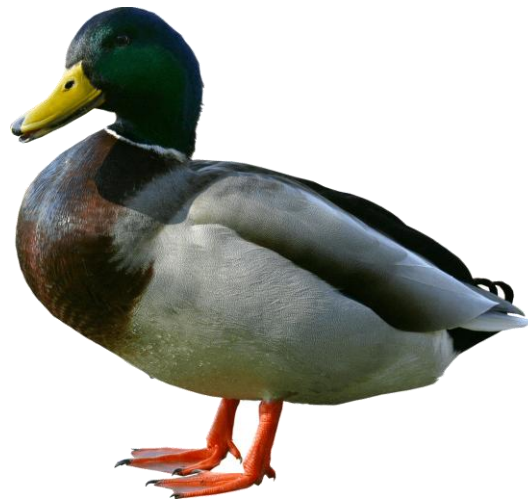


Liskov Substitution Principle



Liskov Substitution Principle

- Subtypes must be **substitutable** for their base types.



Liskov Substitution Principle

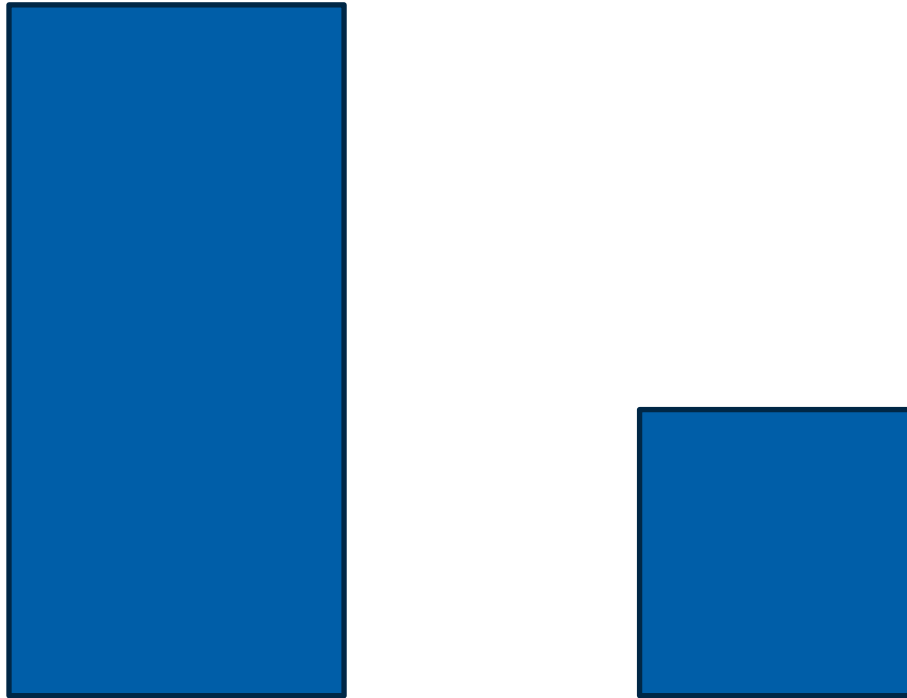
- Something **IS-A** something else
✓ An eagle **IS-A** bird
- Something **HAS-A** property
*An address **HAS-A** city*



LSP states that the **IS-A** relationship is insufficient and should be replaced with **IS-SUBSTITUTABLE-FOR**.



Liskov Substitution Principle



Liskov Substitution Principle

Detecting LSP Violations in Your Code



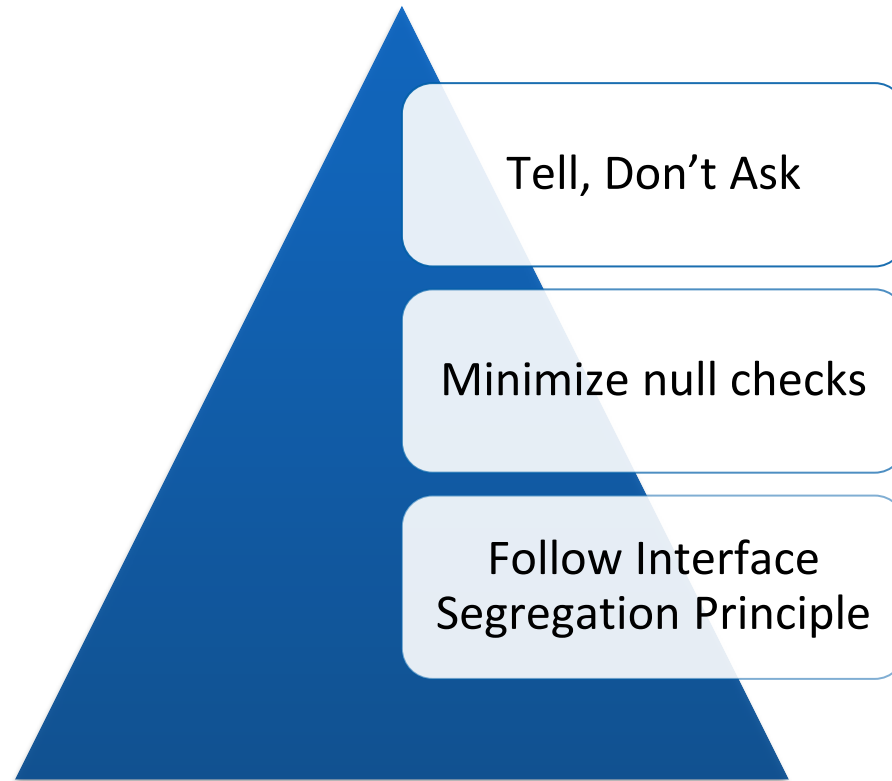
Type checking with `is` or `as` in polymorphic code

Null checks

`NotImplementedException`

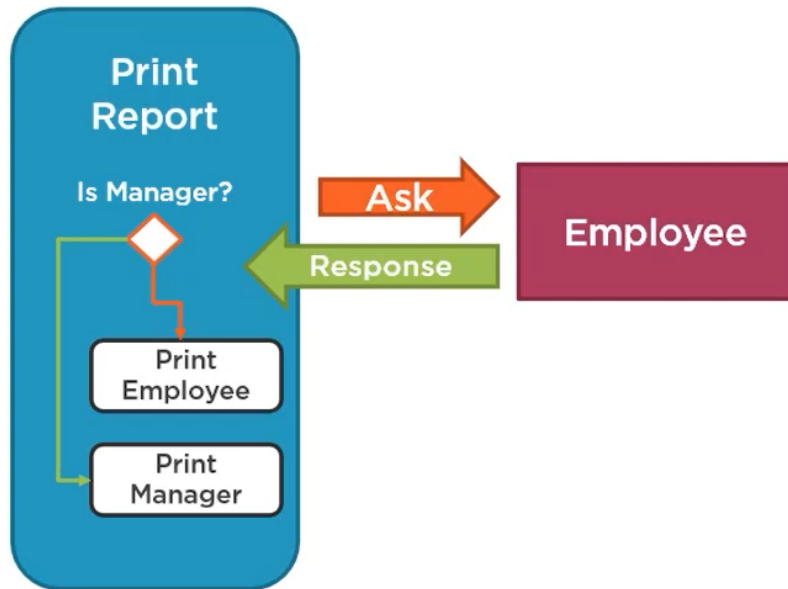
Liskov Substitution Principle

Fixing LSP Violations

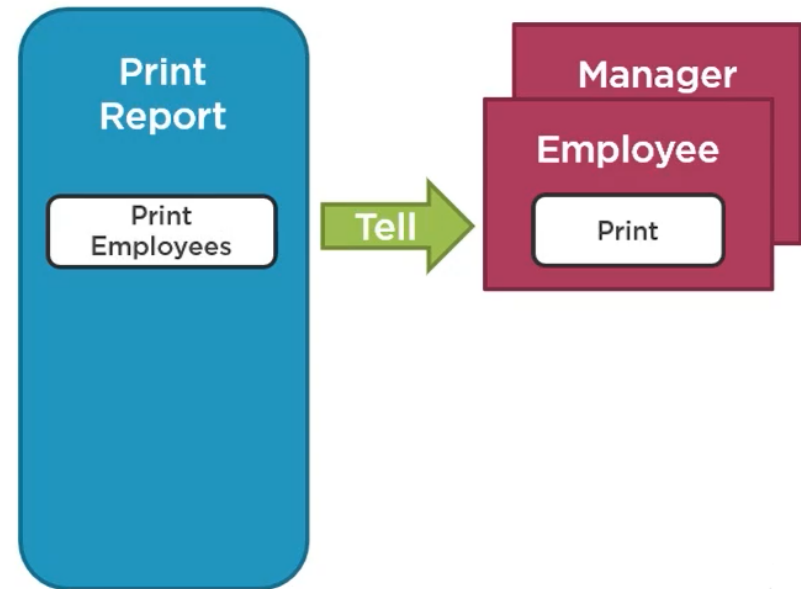


Liskov Substitution Principle

Data and logic separate



Data and logic together

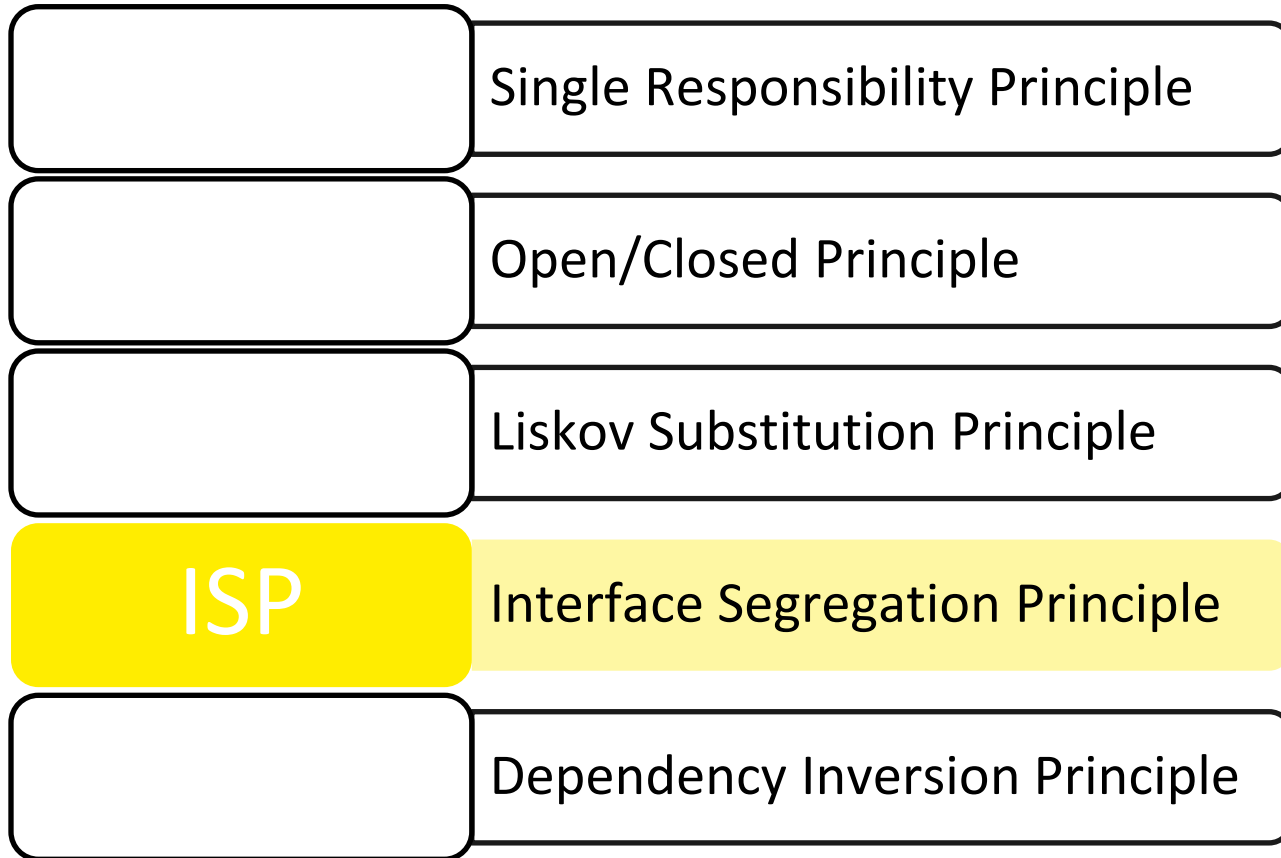


Liskov Substitution Principle

- Don't let the consumer of an abstraction depend on anything but the public contract of that abstraction.
- One object can be designed to inherit from another if it always has an "IS-SUBSTITUTABLE-FOR" relationship with the inherited object.



Interface Segregation Principle



Interface Segregation Principle

- Clients should not be forced to depend on methods they do not use.
- **Client** = the calling code



Interface Segregation Principle

```
abstract SmartHomeModule {  
    turnOnLight(int lightId);  
    turnOffLight(int lightId);  
    isOn(int lightId);  
  
    isOpen(int doorId);  
    openDoor(int doorId);  
    closeDoor(int doorId);  
  
    playMusic(int roomId);  
    pauseMusic(int roomId);  
    ...  
}
```

- ✓ What if all my code needs only the smart lights?
- ✓ Violating ISP results in classes that depend on things they don't need
- ✓ What if I want to implement my own SmartHomeModule?

Interface Segregation Principle

More Dependencies Means

More
coupling

More fragile
code

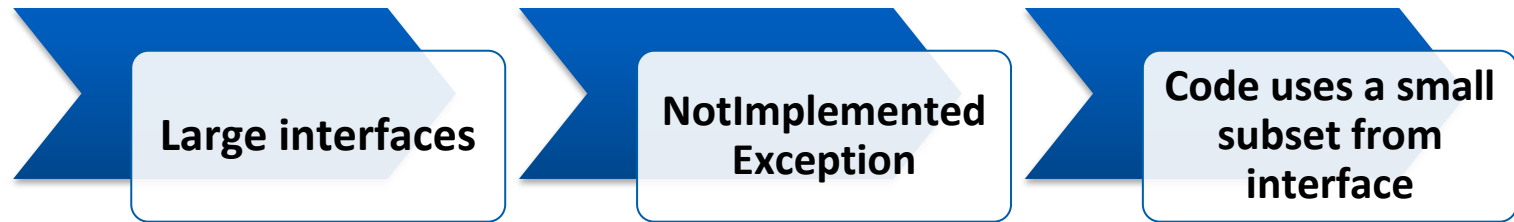
More difficult
testing

More difficult
deployments



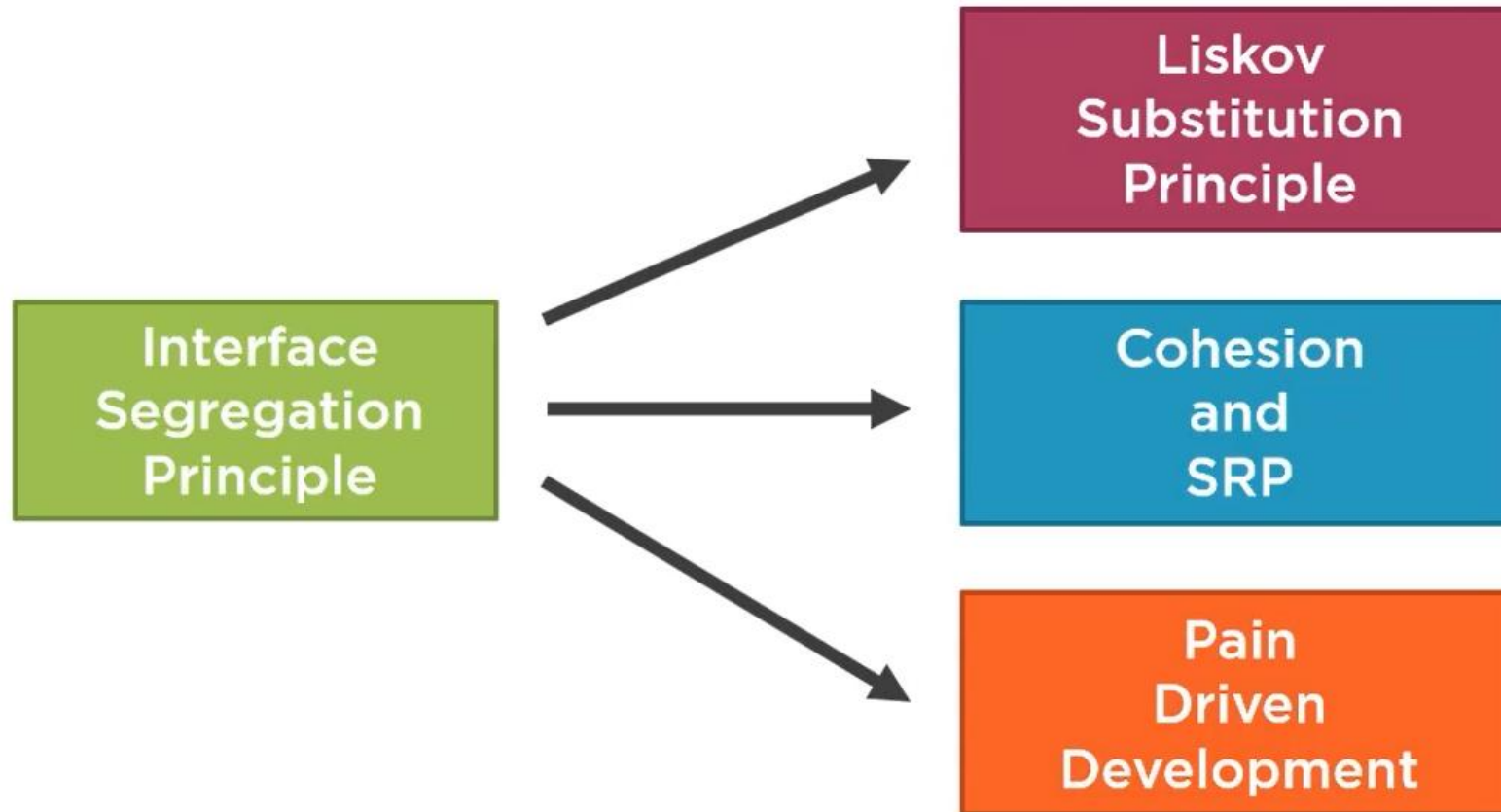
Interface Segregation Principle

Detecting ISP Violations in Your Code



Interface Segregation Principle

Related Concepts



Interface Segregation Principle

Fixing ISP Violations

Break up large interfaces

- Compose large interfaces from smaller ones for backwards compatibility

To address large interfaces you don't control

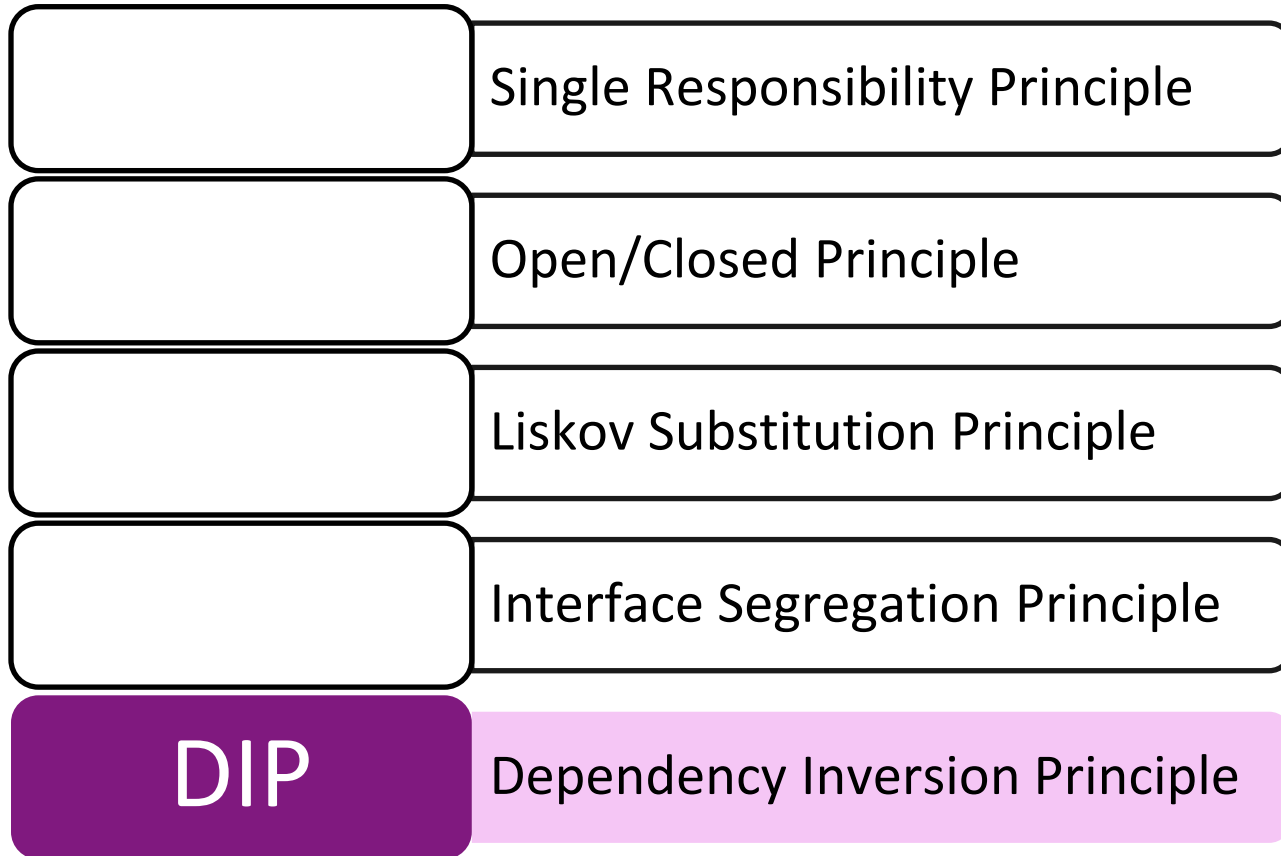
- Create a small/cohesive interface

Interface Segregation Principle

- Each interface should have a specific purpose.
- You shouldn't be forced to implement an interface when your object doesn't share that purpose.
- By extrapolation, the larger the interface, the more likely it includes methods that not all implementers can achieve.
- Ideally, your thin interfaces should be *cohesive*, meaning they have groups of operations that logically belong together.
- This will prevent you from ending up with one-interface-per-method



Dependency Inversion Principle



Dependency Inversion Principle

- High-level modules should not **depend** on low-level modules. Both should **depend** on **abstractions**.
- **Abstractions** should not **depend** on **details**.
- **Details** should **depend** on **abstractions**.



Dependency Inversion Principle

High Level

Process-
oriented

Business rules



Dependency Inversion Principle

Low Level

Closer to I/O

Plumbing
code

Interacts with
external
systems



Dependency Inversion Principle

Abstractions

Interfaces

Abstract base
classes

Types you
can't
instantiate



Dependency Inversion Principle

Details and Abstractions

- **Abstractions** shouldn't be coupled to details
- **Abstractions** describe - the **WHAT**
 - Send a notification
 - Save a record
- **Details** specify – the **HOW**
 - Send a notification via SMS with a specific provider
 - Get the record from a API call and store it into a txt file

Dependency Inversion Principle

Low Level Dependencies



Database



File system



Email



Web APIs



Configuration



Clock

Dependency Inversion Principle

Hidden Direct Dependencies

Direct use of low-level dependencies

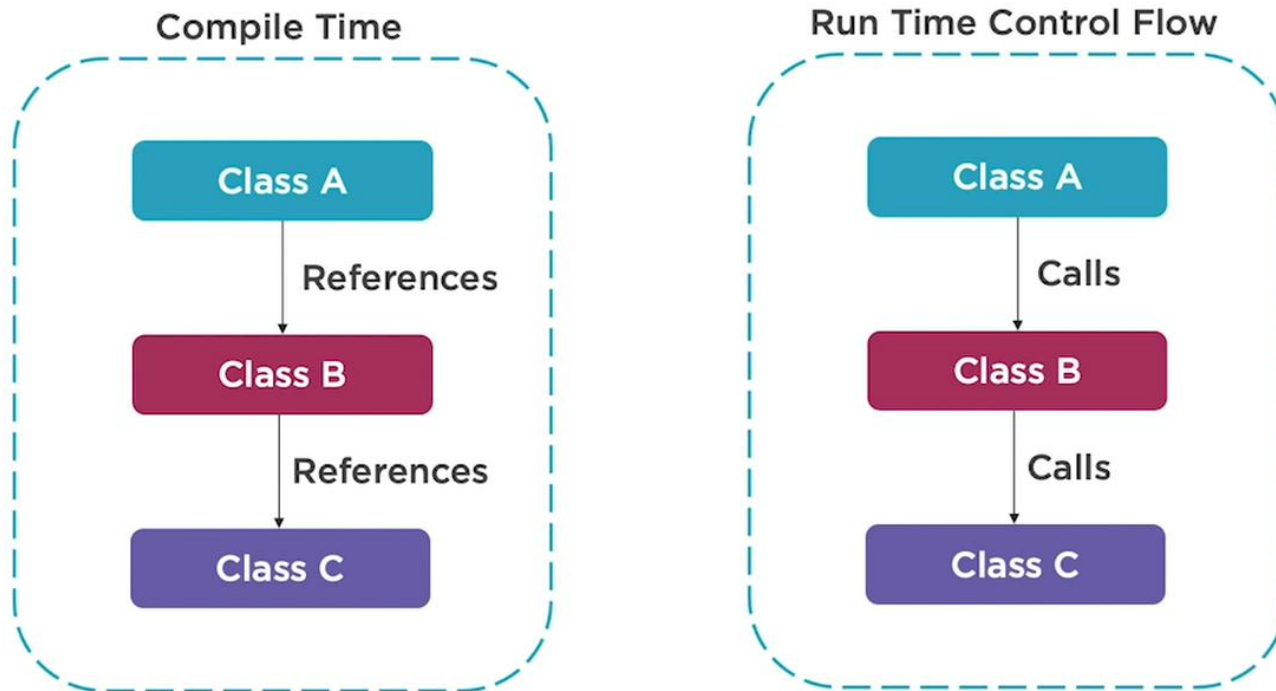
Static call and new

Causes pain

- Tight coupling
- Difficult to unit test

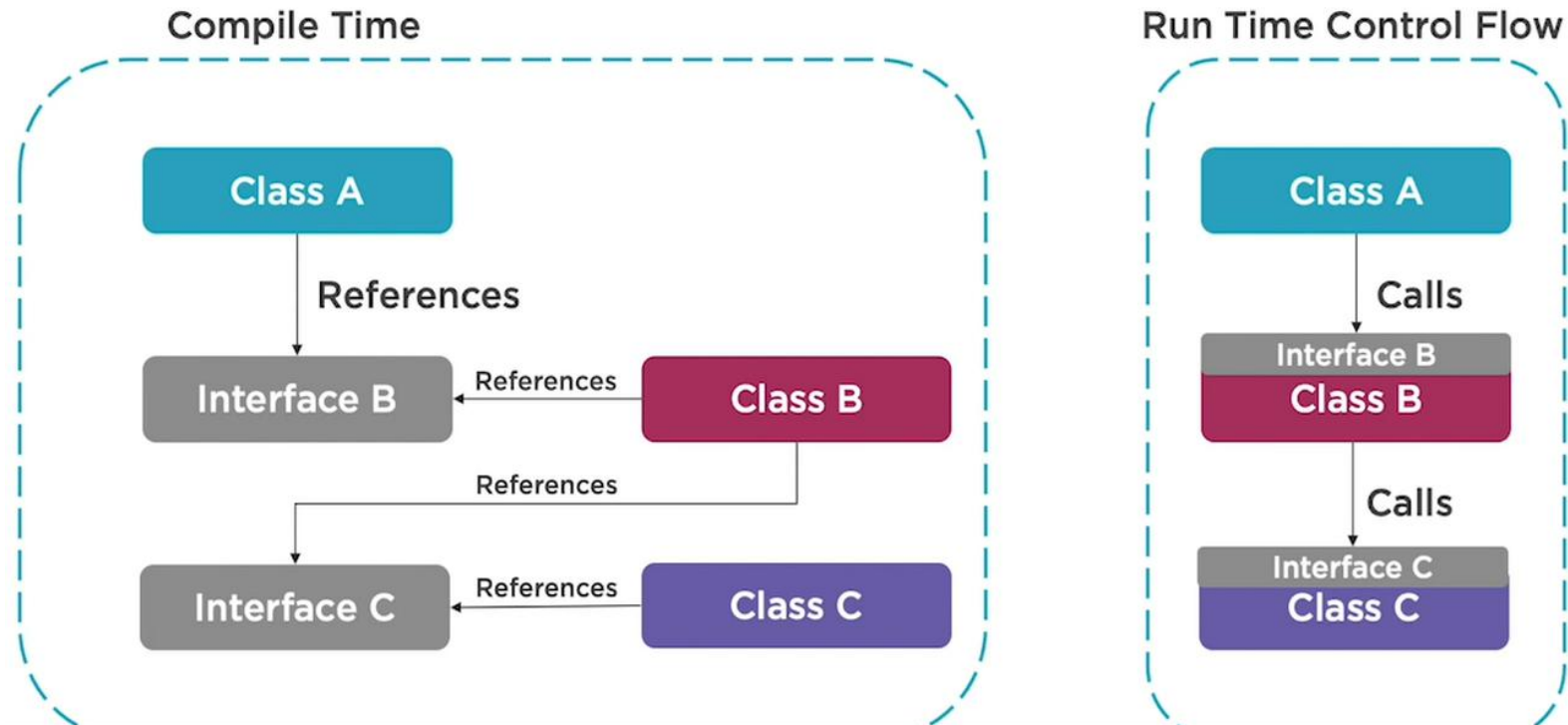
Dependency Inversion Principle

Dependencies without Abstractions



Dependency Inversion Principle

Dependencies **with** Abstractions



Dependency Inversion Principle

Dependency Injection

Don't create your own dependencies

- Depend on abstractions
- Request dependencies from client

Client injects dependencies as

- Constructor arguments ✓
- Properties
- Method arguments



Dependency Inversion Principle

- Methods and classes should explicitly require any collaborating objects they need in order to function correctly
- The constructor contract is telling the client that it only needs the things specified
- By following the explicit dependencies principle, your classes and methods are being honest with their clients about what they need in order to function



